QuakeFloat8 (QF8)

A Provably Optimal Log-Domain 8-Bit Format for ML Multiply-Accumulate

February 2026

Noah Everett

Abstract

QuakeFloat8 (QF8) is a block-scaled logarithmic 8-bit number format for machine learning in which multiplication reduces to 7-bit integer addition. The format uses a 1-bit sign, a 7-bit `u3.4` fixed-point $\log_2$ code per element, and a shared E8M0 block exponent per 32 elements. This document consolidates all theoretical results, hardware cost analysis, training experiments, and literature positioning for the QF8 project.

Key results: (1) Log-uniform quantization is provably minimax-optimal for NMSE over all source distributions (Theorem 3.5). (2) QF8 achieves +6.6 dB SQNR over IEEE FP8 E4M3 at the same effective storage cost. (3) The QF8 multiplier costs ∼66 gates vs. ∼231 for FP8 (3.5× cheaper). (4) TinyGPT-2 training with STE-based QAT matches FP32 validation loss.

Contents

1 Introduction and Motivation **3**
1.1 The 8-Bit Frontier . . . 3
1.2 The Quake Insight . . . 3
1.3 Contributions . . . 3

2 The QF8 Format **3**
2.1 Encoding Scheme . . . 3
2.2 Logarithmic Spacing . . . 4
2.3 Multiplication as Integer Addition . . . 4
2.4 Accumulation Strategy . . . 5
2.5 BMD Realizability . . . 5

3 Theoretical Results: Minimax Optimality **5**
3.1 Notation and Setup . . . 5
3.2 Relation to Classical Quantization Theory . . . 6
3.3 Theorem 2.1: Minimax NMSE Optimality . . . 7

4 Product Error Decomposition **8**
4.1 The General Formula . . . 8
4.2 Centroid Simplification . . . 9
4.3 Specialization to Log-Uniform Quantizers . . . 9
4.4 Optimality for Multiplication . . . 9
4.5 Extension to Dot Products . . . 10

5 Exact MSRE of Log-Uniform Quantization **10**

6 Separation Result: QF8 vs. FP8 **11**
6.1 Exponential MSRE Separation . . . 11
6.2 QF8 vs. FP8 E4M3: Detailed Comparison . . . 12

7 Hardware Cost Analysis **12**
7.1 Reference Data . . . 12
7.2 QF8 Multiply Path . . . 12
7.3 QF8-Medium MAC Unit . . . 12
7.4 Master Comparison . . . 13
7.5 Systolic Array Scaling (128×128) . . . 13
7.6 The Break-Even Analysis . . . 13
7.7 The Honest Bottom Line . . . 13

8 Training Results **14**
8.1 TinyGPT-2 Experiment Configuration . . . 14
8.2 Results . . . 14
8.3 Training Loss Trajectory . . . 14
8.4 Benchmark Results . . . 14

9 Literature Positioning **15**
9.1 Relationship to Johnson (2018) . . . 15
9.2 Relationship to MX Formats (OCP Microscaling) . . . 15
9.3 Relationship to NF4 (QLoRA) . . . 15
9.4 Logarithmic Number Systems . . . 16
9.5 What Is Genuinely Novel . . . 16
9.6 The One-Sentence Pitch . . . 16

10 Future Work **17**
10.1 Scaling Validation . . . 17
10.2 Gradient Quantization . . . 17
10.3 BMD Decoder Conjecture . . . 17

11 Appendix: Rate-Distortion Context **18**
11.1 Gaussian Source Rate-Distortion . . . 18
11.2 Scalar Quantization Gap (Zador) . . . 18
11.3 Bit Allocation . . . 18

Introduction and Motivation

The 8-Bit Frontier

The push toward low-precision arithmetic for deep learning has been remarkably successful. FP16 training is standard practice; BFloat16 is ubiquitous; and 8-bit formats (INT8, FP8 E4M3/E5M2) are deployed in production hardware (NVIDIA H100, AMD MI300, Google TPU). The industry consensus, codified in the OCP Microscaling (MX) specification, is that 8 bits per element with block scaling is sufficient for both training and inference of large language models.

Yet a gap remains between the precision delivered by current 8-bit formats and what is theoretically achievable. FP8 E4M3 provides 8 representable levels per octave (factor of 2). Is this optimal? Can we do better at the same bit budget?

The Quake Insight

In 1999, the Quake III source code revealed a remarkable hack: the integer bit pattern of an IEEE 754 float is approximately a linear function of $\log_2(x)$. Reinterpreting float bits as integers, and vice versa, allows cheap approximate logarithmic arithmetic.

QF8 makes this accident intentional. By storing values as fixed-point $\log_2$ codes, multiplication becomes integer addition — replacing an $O(n^2)$-gate multiplier with an $O(n)$-gate adder. The format is paired with MX-style block scaling (E8M0 shared exponent per 32 elements) to achieve full FP32 dynamic range.

Contributions

1. **Formal optimality.** We prove the *equalization property* of log-uniform quantization: MSRE $= \varepsilon^2/12$ for *any* source density, because the x^2 factors in the NMSE ratio cancel exactly. This density-independence implies that log-uniform quantization is the unique minimax-optimal quantizer for NMSE over all source distributions (Theorem 3.5), complementing Sun & Goyal's [16] variable-rate optimality result and extending it to the fixed-rate minimax setting. Log-uniform quantization also minimizes worst-case product error for multiply-accumulate (Theorem 4.4).

2. **Format design.** QF8 combines a `u3.4` fixed-point log code (16 levels per octave) with E8M0 block scaling, achieving 2× the precision per octave of FP8 E4M3 at the same effective storage cost (8.25 bits/element).

3. **Hardware analysis.** Detailed gate-level models show the QF8 multiplier at ∼66 gates vs. ∼231 for FP8 mantissa multiplication (3.5× cheaper). The complete QF8-Medium MAC unit achieves near-INT8 area and power while providing floating-point dynamic range.

4. **Training validation.** STE-based quantization-aware training on TinyGPT-2 shows QF8 matching FP32 validation loss (2.5445 vs. 2.5450), outperforming FP8 E4M3 (2.5478).

The QF8 Format

Encoding Scheme

Definition 2.1 (QF8 Encoding)**.** A QF8-encoded block of $k = 32$ values consists of:

- **Shared block scale** (8 bits): An E8M0 power-of-2 exponent $s \in \{2^{-127}, \ldots, 2^{127}\}$.
- **Per-element code** (8 bits each): 1 sign bit $\sigma \in \{0, 1\}$ and a 7-bit unsigned `u3.4` fixed-point code $c \in \{0, 1, \ldots, 127\}$.

The reconstruction formula is:

$$\hat{x} = (-1)^{\sigma} \cdot s \cdot 2^{(c-64)/16}, \qquad c \neq 0$$

with $c = 0$ encoding exact zero. The effective storage cost is $8 + 8/32 = 8.25$ bits per element.

Logarithmic Spacing

The `u3.4` code provides 3 integer bits and 4 fractional bits of $\log_2 |x/s|$, spanning 8 octaves with **16 representable levels per octave**. This is the *log-uniform quantizer*: the ratio between consecutive representable values is constant at $r = 2^{1/16} \approx 1.0443$, giving a maximum relative quantization error of

$$\frac{r-1}{2} \approx 2^{1/16} - 1 \approx 4.43\%.$$

Compare FP8 E4M3, which has 8 levels per octave (3-bit mantissa) and a maximum relative error of $1/16 = 6.25\%$ within each binade.

Remark (Why `u3.4`: Bit Allocation). The 7-bit log code admits several integer/fractional splits. The choice of 3 integer + 4 fractional bits is determined by the ML workload:

- **Per-element range:** 3 integer bits give $2^3 = 8$ octaves of per-element dynamic range. With E8M0 block scaling absorbing inter-block variation, the per-element code only needs to cover *within-block* spread. For typical ML tensors with block size 32, empirical within-block spread is 3–5 octaves; 8 octaves provides comfortable margin.
- **Precision:** 4 fractional bits give 16 levels per octave (MSRE $= 1.56 \times 10^{-4}$). Shifting one fractional bit to integer (`u4.3`: 16 octaves, 8 levels/octave) halves precision for already-sufficient range — strictly worse for weights and activations. Shifting one integer bit to fractional (`u2.5`: 4 octaves, 32 levels/octave) risks overflow for blocks with > 4 octave spread.
- **Gradient variant:** For backward passes, where gradients require wider dynamic range, a `u4.3` variant (16 octaves, 8 levels/octave) is the natural analog of FP8's E5M2 for gradients (Section 10).

Multiplication as Integer Addition

For two QF8 values with log codes c_a, c_b:

$$\text{sign}_{\text{product}} = \sigma_a \oplus \sigma_b \tag{1}$$

$$\text{log-code}_{\text{product}} = c_a + c_b \tag{2}$$

The product's log code is a simple 7-bit integer addition. In hardware, this replaces the 4×4 mantissa multiplier needed for FP8 E4M3 with a 7-bit carry-lookahead adder — a $\sim$3.5$\times$ reduction in gate count (66 vs. 231 gates).

Accumulation Strategy

The classical weakness of log-domain arithmetic is that *addition* in the original domain is expensive. QF8 adopts the **Kulisch accumulation** strategy (Johnson, 2018):

1. Multiply in log domain (integer addition of codes).
2. Convert each product to linear domain via a tiny $2^4 = 16$-entry lookup table.
3. Accumulate in a wide fixed-point register (32–48 bits for practical designs).

This hybrid log-multiply / linear-accumulate approach gets cheap multiplies *and* exact (or near-exact) accumulation.

BMD Realizability

Definition 2.2 (Bit-Manipulation Decodable (BMD))**.** A decoder $\hat{D} : \{0,1\}^B \to \mathbb{R}$ is *bit-manipulation decodable* if it can be expressed as a fixed-length straight-line program over $\{+, -, \times, \gg, \ll, \&, |, \oplus\}$ on machine-word integers, with the output reinterpreted as a float via type punning.

Theorem 2.3 (BMD Realizability)**.** *The log-uniform decode function $\hat{D}(n) = a \cdot r^n$ can be implemented via:*

1. ***Lookup table:*** *$O(N)$ storage, $O(1)$ read.*
2. ***Reinterpret cast (Quake trick):*** *$I = \alpha n + \beta$, then float reinterpret. Cost: 1 multiply + 1 add + type pun = 3 ops.*
3. ***Quadratic approximation:*** *$2^{e+f} = 2^e \cdot (1 + 0.6602f + 0.3398f^2)$. Cost: 2 multiplies + 2 adds + 1 shift = 5 ops. Maximum relative error: 0.27% (minimax optimal for constrained quadratic).*

Theoretical Results: Minimax Optimality

Notation and Setup

Let X be a positive random variable with pdf f supported on $[a, b] \subset \mathbb{R}_{>0}$. A B-bit scalar quantizer is a map $Q : [a, b] \to \hat{\mathcal{X}}$ with $N = 2^B$ reconstruction points partitioning $[a, b]$ into cells $\{S_k\}_{k=0}^{N-1}$ with codepoints $\{c_k\}$.

Definition 3.1 (Distortion Measures)**.**

$$\delta_X = X - Q(X), \qquad \varepsilon_X = \delta_X / X \tag{3}$$

$$\mathrm{MSE}_X = \mathbb{E}[\delta_X^2], \qquad \mathrm{NMSE}_X = \frac{\mathbb{E}[\delta_X^2]}{\mathbb{E}[X^2]} \tag{4}$$

$$\mathrm{MSRE}_X = \mathbb{E}[\varepsilon_X^2], \qquad \mathrm{SQNR} = 10 \log_{10}(1/\mathrm{NMSE}) \text{ dB} \tag{5}$$

Definition 3.2 (Auxiliary Quantities)**.**

$$\alpha_X = \mathbb{E}[X\delta_X]/\mathbb{E}[X^2] \qquad \text{(signal-error correlation)} \tag{6}$$

$$\beta_X = \mathbb{E}[Q(X)\delta_X]/\mathbb{E}[X^2] \qquad \text{(reconstruction-error correlation)} \tag{7}$$

$$\gamma_X = \mathbb{E}[Q(X)^2]/\mathbb{E}[X^2] \qquad \text{(quantized power ratio)} \tag{8}$$

$$\rho_X = \mathbb{E}[X \cdot Q(X)]/\mathbb{E}[X^2] \qquad \text{(signal-quantized correlation)} \tag{9}$$

Algebraic identities (no assumptions required):

$$\alpha_X = \beta_X + \mathrm{NMSE}_X, \qquad \gamma_X = 1 - 2\alpha_X + \mathrm{NMSE}_X, \qquad \rho_X = 1 - \alpha_X \tag{10}$$

Definition 3.3 (Centroid Condition)**.** A quantizer satisfies the centroid condition if $\mathbb{E}[\delta_X \mid X \in S_k] = 0$ for every cell. This holds for Lloyd-Max quantizers and approximately for any well-designed quantizer in the high-resolution regime. Under the centroid condition: $\beta_X = 0$, $\alpha_X = \mathrm{NMSE}_X$, and $\gamma_X = 1 - \mathrm{NMSE}_X$.

Definition 3.4 (Log-Uniform Quantizer)**.** The log-uniform quantizer with N levels over dynamic range $[a, b]$ has cell boundaries $a \cdot r^k$ for $k = 0, \ldots, N$, where $r = (b/a)^{1/N} = 2^{R/N}$ and $R = \log_2(b/a)$ is the dynamic range in octaves. With geometric midpoint codepoints $c_k = a \cdot r^{k+1/2}$, the relative cell width is constant:

$$\frac{w(x)}{x} = r - 1 = e^{\varepsilon} - 1 \approx \varepsilon, \qquad \varepsilon = \frac{R \ln 2}{N} \tag{11}$$

Relation to Classical Quantization Theory

The classical companding literature studies optimal nonuniform quantizers under MSE distortion. Bennett [13] established the foundational high-resolution framework: for a compander with local step size $w(x)$ and source pdf f, the distortion is $D = \frac{1}{12} \int w(x)^2 f(x)\, dx$, giving the $\varepsilon^2/12$ approximation for uniform quantizers. Panter & Dite [14] derived the MSE-optimal compander (the $p(x)^{1/3}$ one-third-power rule): it achieves equal MSE contribution per cell, but depends critically on the source distribution. The standard textbook treatments [17, 18] cover these results thoroughly.

For *relative* error metrics, Smith [15] observed that logarithmic companding achieves approximately amplitude-independent SNR for PCM telephony — meaning the SNR is roughly constant as a function of *signal level* (e.g., sinusoidal amplitude). This is structurally distinct from the equalization property of Theorem 3.5: Smith's result is about signal amplitude, not about source probability density. Indeed, Smith's distortion formula retains a distribution-dependent factor (e.g., 0.707 for exponential, 0.798 for Gaussian distributions), so the distortion *is* distribution-dependent in his framework.

Most directly related, Sun & Goyal [16] study scalar quantization explicitly under the MSRE metric. They prove that logarithmic companding is *variable-rate*-optimal for MSRE: given a source distribution f, the entropy-constrained optimal quantizer is logarithmic. However, they also show that logarithmic companding is *not* generally optimal for fixed-rate quantization — the optimal fixed-rate MSRE quantizer depends on the source distribution.

Theorem 3.5 provides a complementary and stronger fixed-rate result: the log-uniform quantizer achieves MSRE $= \varepsilon^2/12$ for *any* source density (the equalization property), and this constant value is the fixed-rate minimax optimum. The key structural insight is the x^2 cancellation in the NMSE ratio, which is absent in all prior companding results: it makes the log-uniform quantizer's error density-independent rather than merely amplitude-independent. The minimax framing is not addressed in any prior work found.

Why minimax for ML? The ML setting motivates worst-case (minimax) rather than average-case (Bayes) analysis for two reasons. First, weight and activation distributions shift substantially across layers, training stages, and models, so a quantizer optimal for one distribution may be poor for another. Second, the equalization property has direct practical value: a format with density-independent NMSE requires no per-layer calibration of the quantizer grid, and its error bound holds regardless of whether the layer distribution matches any assumed prior. The log-uniform quantizer's minimax value $\varepsilon^2/12$ is not just a worst-case bound — by the equalization property, it

is achieved for every source distribution, including Gaussian, uniform, Laplace, and the empirical distributions observed in trained networks.

Theorem 2.1: Minimax NMSE Optimality

Theorem 3.5 (Minimax NMSE Optimality of Log-Uniform Quantization)**.** *In the high-resolution regime ($N \gg 1$), among all N-level quantizers on $[a, b]$, the log-uniform quantizer uniquely minimizes the worst-case NMSE (and MSRE) over all densities:*

$$Q^*_{\log} = \arg\min_{Q \in \mathcal{Q}_N} \max_{f \in \mathcal{F}[a,b]} \mathrm{NMSE}(Q, f)$$

The minimax value is:

$$\boxed{\mathrm{NMSE}^* = \frac{\varepsilon^2}{12} + O(\varepsilon^4) = \frac{(R \ln 2)^2}{12N^2}} \tag{12}$$

For $B = 8$ bits, $R = 16$ octaves: $\mathrm{NMSE}^* = 1.564 \times 10^{-4}$ *(*$\mathrm{SQNR} = 38.1$ *dB).*

Proof. **Step 1 (Density-independence).** For the log-uniform quantizer, $w(x) = x \cdot \varepsilon + O(x\varepsilon^2)$. Under the high-resolution approximation:

$$\mathrm{NMSE} = \frac{\int_a^b \frac{w(x)^2}{12} f(x)\, dx}{\int_a^b x^2 f(x)\, dx} = \frac{\frac{\varepsilon^2}{12} \int_a^b x^2 f(x)\, dx}{\int_a^b x^2 f(x)\, dx} = \frac{\varepsilon^2}{12}$$

The x^2 factors cancel exactly. Similarly, $\mathrm{MSRE} = \varepsilon^2/12$ for any density f. This is the *equalization property* of log-uniform quantization.

This is confirmed by exhaustive computation over three test densities (log-uniform, uniform, $f \propto x^2$) with $N = 256$, $R = 16$: all give $\mathrm{NMSE} = 1.564 \times 10^{-4}$ within $< 0.1\%$ deviation.

Step 2 (Non-constant ϕ is worse for some density). Define $\phi(x) = w(x)/x$ (relative cell width). For any non-log-uniform quantizer, ϕ is not constant. Let $x^* = \arg\max_x \phi(x)$ and $\phi^* = \phi(x^*)$.

We first show $\phi^* > \varepsilon$. Any N-cell quantizer on $[a, b]$ satisfies the coverage constraint $\int_a^b dx/w(x) = N$, i.e.

$$\int_a^b \frac{dx}{\phi(x)\, x} = N. \tag{13}$$

For the log-uniform quantizer $\phi \equiv \varepsilon$, this gives $(1/\varepsilon) \ln(b/a) = N$ since $\varepsilon = R \ln 2/N$ and $\ln(b/a) = R \ln 2$. Suppose for contradiction that ϕ is non-constant but $\phi(x) \le \varepsilon$ everywhere. Then $1/\phi(x) \ge 1/\varepsilon$ everywhere, with strict inequality on a set of positive measure, so

$$N = \int_a^b \frac{dx}{\phi(x)\, x} > \int_a^b \frac{dx}{\varepsilon\, x} = \frac{\ln(b/a)}{\varepsilon} = N,$$

a contradiction. Therefore $\phi^* > \varepsilon$ for any non-log-uniform quantizer.

For any $\eta > 0$, the uniform density on $[x^* - \eta, x^* + \eta]$ achieves:

$$\mathrm{MSRE} \ge \frac{\phi(x^*)^2}{12} - g(\eta), \qquad g(\eta) \to 0 \text{ as } \eta \to 0$$

Therefore $\max_f \mathrm{MSRE}(Q, f) \ge (\phi^*)^2/12 > \varepsilon^2/12 = \mathrm{MSRE}^*$.

Step 3 (Uniqueness). If $Q \ne Q_{\log}$, then ϕ is non-constant and Step 2 gives strict inequality. □ □

Corollary 3.6 (Minimax MSRE)**.** *The same quantizer also minimizes worst-case MSRE. The proof is identical, with $w(x)^2 f(x)/x^2$ replacing $w(x)^2 f(x)/\mathbb{E}[X^2]$.*

Remark (High-Resolution Regime)*.* The proof uses Bennett's high-resolution integral approximation, which is asymptotically exact as $N \to \infty$. For finite N, Theorem 5.1 provides the exact (non-asymptotic) per-cell MSRE: the ratio of exact to approximate MSRE is 0.9997 at $N = 256$ (Table 1). The minimax conclusion holds for all practical quantizer sizes ($N \geq 16$).

Remark (Domain Restriction and Zero Handling)*.* Theorem 3.5 applies to densities on $[a, b] \subset \mathbb{R}_{>0}$, excluding zero. In practice, ML tensors contain exact zeros (e.g., ReLU outputs) and near-zero values. QF8 handles this in two ways: (1) the code $c = 0$ represents exact zero, absorbing any point mass at the origin; (2) block scaling with E8M0 exponents as small as 2^{-127} shifts the smallest nonzero representable value to $\sim 5.9 \times 10^{-39}$, far below any practical threshold. The minimax guarantee applies to the nonzero portion of the signal; the zero code handles the rest exactly.

Product Error Decomposition

This section addresses how quantization error propagates through multiplication — the core operation in neural network inference.

The General Formula

Theorem 4.1 (General Product Error Decomposition)**.** *Let X, Y be independent positive random variables with quantizers Q_X, Q_Y. Then:*

$$\boxed{\mathrm{NMSE}_{\mathrm{prod}} = \mathrm{NMSE}_X + \mathrm{NMSE}_Y + \mathrm{NMSE}_X \cdot \mathrm{NMSE}_Y + 2\alpha_X\alpha_Y - 2\alpha_X \cdot \mathrm{NMSE}_Y - 2\mathrm{NMSE}_X \cdot \alpha_Y} \tag{14}$$

where $\alpha_X = \mathbb{E}[X\delta_X]/\mathbb{E}[X^2]$. Equivalently:

$$\mathrm{NMSE}_{\mathrm{prod}} = 1 - 2\rho_X\rho_Y + \gamma_X\gamma_Y \tag{15}$$

This is exact, using only $X \perp Y$. No high-resolution, centroid, or distributional assumptions.

Proof. **Step 1.** Decompose the product error:

$$XY - Q_XQ_Y = (Q_X + \delta_X)(Q_Y + \delta_Y) - Q_XQ_Y = \underbrace{Q_X\delta_Y}_{A} + \underbrace{\delta_XQ_Y}_{B} + \underbrace{\delta_X\delta_Y}_{C}$$

Step 2. Expand $\mathbb{E}[(A + B + C)^2]$ using $X \perp Y$. Since Q_X, δ_X depend only on X and Q_Y, δ_Y depend only on Y, every product of an X-function and a Y-function factors:

$$\begin{aligned}\mathbb{E}[(A + B + C)^2] = {} & \mathbb{E}[Q_X^2]\mathbb{E}[\delta_Y^2] + \mathbb{E}[Q_Y^2]\mathbb{E}[\delta_X^2] + \mathbb{E}[\delta_X^2]\mathbb{E}[\delta_Y^2] \\ & + 2\mathbb{E}[Q_X\delta_X]\mathbb{E}[Q_Y\delta_Y] + 2\mathbb{E}[Q_X\delta_X]\mathbb{E}[\delta_Y^2] + 2\mathbb{E}[\delta_X^2]\mathbb{E}[Q_Y\delta_Y]\end{aligned}$$

Step 3. Divide by $\mathbb{E}[X^2]\mathbb{E}[Y^2]$ and substitute $\beta_X = \alpha_X - \mathrm{NMSE}_X$, $\gamma_X = 1 - 2\alpha_X + \mathrm{NMSE}_X$:

$$\mathrm{NMSE}_{\mathrm{prod}} = \gamma_X\mathrm{NMSE}_Y + \gamma_Y\mathrm{NMSE}_X + \mathrm{NMSE}_X\mathrm{NMSE}_Y + 2\beta_X\beta_Y + 2\beta_X\mathrm{NMSE}_Y + 2\mathrm{NMSE}_X\beta_Y$$

Expanding $\beta = \alpha - \mathrm{NMSE}$ and simplifying yields (14). □ □

Centroid Simplification

Corollary 4.2 (Product Error Under Centroid Condition)**.** *If both quantizers satisfy the centroid condition ($\alpha_X = \mathrm{NMSE}_X$, $\alpha_Y = \mathrm{NMSE}_Y$), then:*

$$\boxed{1 - \mathrm{NMSE}_{\mathrm{prod}} = (1 - \mathrm{NMSE}_X)(1 - \mathrm{NMSE}_Y)} \tag{16}$$

Equivalently: $\mathrm{NMSE}_{\mathrm{prod}} = \mathrm{NMSE}_X + \mathrm{NMSE}_Y - \mathrm{NMSE}_X \cdot \mathrm{NMSE}_Y$.

Proof. Substitute $\alpha_X = n_X$, $\alpha_Y = n_Y$ into (14): $n_X + n_Y + n_X n_Y + 2n_X n_Y - 2n_X n_Y - 2n_X n_Y = n_X + n_Y - n_X n_Y$. □ □

Remark (Sign of the Cross-Term). The correct sign is **minus** ($n_X + n_Y - n_X n_Y$), not plus as in the original formulation. The identity $(1 - n_X)(1 - n_Y) = 1 - n_{\mathrm{prod}}$ has a clean interpretation: the signal preservation fraction of the product equals the product of individual preservation fractions. However, since $n_X n_Y = O(1/N^4)$ while $n_X + n_Y = O(1/N^2)$, the distinction is negligible in practice. The corrected sign and the equivalence of the two forms (14) and (15) were confirmed by exhaustive computation over 14,641 integer grid points covering all combinations of $(\mathrm{NMSE}_X, \mathrm{NMSE}_Y) \in \{0, 0.01, \ldots, 1\}^2$.

Remark (Correction from Original). The original Theorem 3.1 used an MSRE-based formula with an invalid factorization ($\mathbb{E}[X^2\varepsilon_X^2] = \mathbb{E}[X^2]\mathbb{E}[\varepsilon_X^2]$, which fails because ε_X depends deterministically on X). For non-log-uniform quantizers, the MSRE formula can be off by factors of 6–1000×. The corrected formula (14) uses only the valid factorization $X \perp Y$.

Specialization to Log-Uniform Quantizers

Theorem 4.3 (Log-Uniform Product Error)**.** *For log-uniform quantizers with geometric midpoints,* $\mathrm{MSRE} = \mathrm{NMSE} = \varepsilon^2/12$ *for any density, and:*

$$\mathrm{NMSE}_{\mathrm{prod}} = \frac{(R \ln 2)^2}{6N^2} + O(1/N^4) = 2\mathrm{NMSE}^* + O(1/N^4) \tag{17}$$

The original MSRE-based formula gives the correct answer to leading order for log-uniform quantizers (despite being invalid in general) because the equalization property makes $\mathrm{MSRE} \approx \mathrm{NMSE}$ *with ratio* 1.0009 *for* $N = 16$.

Optimality for Multiplication

Theorem 4.4 (Log-Uniform Optimal for Multiplication)**.** *Among all N-level centroid-satisfying quantizer pairs (Q_X, Q_Y) on $[a, b]$, the log-uniform quantizer minimizes the worst-case product NMSE:*

$$Q_X^* = Q_Y^* = Q_{\log} = \operatorname*{arg\,min}_{Q_X, Q_Y \in \mathcal{Q}_N^c} \max_{f_X, f_Y} \mathrm{NMSE}_{\mathrm{prod}}$$

where $\mathcal{Q}_N^c$ denotes centroid-satisfying N-level quantizers. The minimax value is $2n^* - (n^*)^2 = 1 - (1 - n^*)^2$ where $n^* = \varepsilon^2/12$.

Proof. Uses two properties: (1) *Monotonicity*: under the centroid condition, $\mathrm{NMSE}_{\mathrm{prod}} = n_X + n_Y - n_X n_Y$, and $\partial/\partial n_X (n_X + n_Y - n_X n_Y) = 1 - n_Y > 0$. (2) *Density-independence*: $Q_{\log}$ achieves $\mathrm{NMSE} = n^*$ for all densities. For any $Q_X \neq Q_{\log}$, Theorem 3.5 provides f_X^* with $\mathrm{NMSE}(Q_X, f_X^*) > n^*$, giving strictly worse product NMSE. □

Remark (Centroid Restriction). The centroid condition ($\mathbb{E}[\delta_X \mid X \in S_k] = 0$) is without loss of generality for this optimization: given any quantizer Q that violates it, replacing each codepoint by its conditional mean yields a quantizer Q' with strictly lower NMSE and the same cell partition. The minimax over centroid quantizers therefore equals the minimax over all quantizers.

Extension to Dot Products

Corollary 4.5 (Dimension-Free Dot Product NMSE)**.** *For $Z = \sum_{i=1}^{d} w_i x_i$ with independent pairs, iid within type, and centroid-condition quantizers:*

$$\mathrm{NMSE}_Z \approx \mathrm{NMSE}_w + \mathrm{NMSE}_x + O(1/N^4) \tag{18}$$

The NMSE does not grow with dimension d.

Proof sketch. Under independence across dimensions: $\mathbb{E}[(Z - \hat{Z})^2] = d \cdot \mathbb{E}[w^2 x^2](n_w + n_x - n_w n_x)$ and $\mathbb{E}[Z^2] = d \cdot \mathbb{E}[w^2 x^2]$. Division yields the result. □

Exact MSRE of Log-Uniform Quantization

Theorem 5.1 (Exact Per-Cell MSRE)**.** *For a log-uniform quantizer cell with common ratio $r = 2^{R/N}$ and geometric midpoint codepoint, the exact MSRE (under uniform intra-cell density) is:*

$$\mathrm{MSRE}_k = 2 - \frac{2\sqrt{r}\,\ln r}{r-1} \tag{19}$$

For small $\varepsilon = \ln r = R \ln 2/N$:

$$\boxed{\mathrm{MSRE}_k = \frac{\varepsilon^2}{12} - \frac{7\varepsilon^4}{2880} + O(\varepsilon^6)} \tag{20}$$

Proof. For a cell $[a, ra)$ with geometric midpoint $c = a\sqrt{r}$, substituting $u = x/a$:

$$\mathrm{MSRE}_k = \frac{1}{r-1}\int_1^r \left(1 - \frac{\sqrt{r}}{u}\right)^2 du = \frac{2(r-1) - 2\sqrt{r}\ln r}{r-1} = 2 - \frac{2\sqrt{r}\,\ln r}{r-1}$$

For the Taylor expansion, substitute $r = e^{\varepsilon}$:

$$\frac{2\sqrt{r}\,\ln r}{r-1} = \frac{2\varepsilon\, e^{\varepsilon/2}}{e^{\varepsilon} - 1} = \frac{2(1 + \varepsilon/2 + \varepsilon^2/8 + \cdots)}{1 + \varepsilon/2 + \varepsilon^2/6 + \cdots} = 2\left(1 - \frac{\varepsilon^2}{24} + O(\varepsilon^3)\right)$$

Therefore $\mathrm{MSRE}_k = 2 - (2 - \varepsilon^2/12) = \varepsilon^2/12 + O(\varepsilon^4)$. This *confirms* the high-resolution approximation. □ □

Remark (Correction of $\varepsilon^2/2$ Error). The original derivation approximated $e^{\varepsilon} - 1 \approx \varepsilon$, omitting the $\varepsilon^2/2$ correction in the denominator. The correct denominator is $\varepsilon(1 + \varepsilon/2 + O(\varepsilon^2))$; this extra factor converts the erroneous $\varepsilon^2/2$ to the correct $\varepsilon^2/12$ — a factor of 6 correction.

Table 1: Numerical verification: exact MSRE vs. approximations.

ε	Exact MSRE	$\varepsilon^2/12$	$\varepsilon^2/2$ (wrong)	Exact / ($\varepsilon^2/12$)
0.01	8.333×10^{-6}	8.333×10^{-6}	5.000×10^{-5}	1.000
0.05	2.083×10^{-4}	2.083×10^{-4}	1.250×10^{-3}	0.9999
0.10	8.331×10^{-4}	8.333×10^{-4}	5.000×10^{-3}	0.9997
0.20	3.329×10^{-3}	3.333×10^{-3}	2.000×10^{-2}	0.9989

Separation Result: QF8 vs. FP8

Exponential MSRE Separation

Theorem 6.1 (Exponential Separation Under MSRE)**.** *For $X \sim \mathrm{LogNormal}(0, \sigma^2)$ quantized to N levels:*

(i) ***Under MSRE:*** *the ratio of uniform quantization MSRE to logarithmic quantization MSRE is exponential in σ^2:*

$$\frac{\mathrm{MSRE}_{\mathrm{uniform}}}{\mathrm{MSRE}_{\log}} = e^{\Theta(\sigma^2)} \tag{21}$$

(ii) ***Under MSE:*** *the two quantizers achieve comparable distortion (ratio ≈ 1).*

This result is significant because MSRE is the natural metric for floating-point systems — it measures relative accuracy, which is what determines model quality.

Table 2: MSRE and MSE ratios (uniform / log) for $\mathrm{LogNormal}(0, \sigma^2)$, $N = 256$.

σ	MSE ratio	MSE dB	MSRE ratio	MSRE dB
1.00	0.98	-0.1	8.5	9.3
1.50	0.99	-0.0	1,520	31.8
2.00	0.99	-0.0	346,000	55.4
2.77	1.00	0.0	3.0×10^9	94.8

Proof sketch. For $X = e^Z$ with $Z \sim \mathcal{N}(0, \sigma^2)$: Log quantization on X corresponds to uniform quantization on Z (Gaussian), giving $\mathrm{MSRE}_{\log} \approx \varepsilon^2/12$. Uniform quantization on X translates to highly nonuniform quantization on $Z = \ln X$: at $Z = k\sigma$, the local step in log-space grows as $\propto e^{2k\sigma}$, and integrating against the Gaussian density yields $\mathrm{MSRE}_{\mathrm{uniform}} \propto e^{c\sigma^2}$. MSE is insensitive because it is dominated by absolute error on large values where both quantizers are comparable. □

Table 3: Within-binade precision comparison.

Metric	IEEE E4M3	Log-uniform (8/oct)	Ratio	dB gap
Average MSRE (per binade)	6.510×10^{-4}	6.256×10^{-4}	1.041	0.2
Peak relative error	0.0625	0.04427	1.41	—
Worst-case MSRE (HR)	1.302×10^{-3}	6.256×10^{-4}	2.08	3.18

QF8 vs. FP8 E4M3: Detailed Comparison

Key Result

QF8 achieves SQNR = 38.1 dB vs. FP8 E4M3's 31.5 dB on $\mathcal{N}(0,1)$ — a **6.6 dB advantage**, corresponding to a ∼4.6× reduction in quantization noise power. This advantage is structural (16 vs. 8 levels per octave) and holds across all tested distributions.

Format	SQNR on $\mathcal{N}(0,1)$	Gap from QF8
QF8 (log-uniform, 256 levels)	38.1 dB	—
FP8 E4M3 (IEEE)	31.5 dB	6.6 dB worse

Remark. The FP8 E4M3 SQNR of 31.5 dB is verified by per-binade analysis across all 126 positive representable values, correcting an earlier figure of 25.1 dB that arose from an error in the dynamic-range assumption.

Hardware Cost Analysis

This section presents gate-level hardware models anchored to published silicon data from Horowitz (ISSCC 2014), Johnson (2018 ELMA ASIC at 28nm), and NVIDIA architecture whitepapers.

Reference Data

Horowitz ISSCC 2014 (45nm CMOS): 8-bit integer multiply = 0.2 pJ, 8-bit add = 0.03 pJ, 32-bit FP multiply = 3.7 pJ.

Johnson ELMA synthesis (28nm): 8-bit ELMA/38-bit Kulisch = 0.96× power, 1.12× area vs. INT8/32-bit MAC. 16-bit ELMA = 0.59× power, 0.68× area vs. FP16 FMA.

QF8 Multiply Path

The core QF8 advantage: multiplication reduces to a 7-bit integer addition plus sign XOR.

Compare: INT8 8×8 multiplier = 400 gates; FP8 4×4 mantissa multiplier + exponent logic = 230 gates; FP16 11×11 multiplier = 850 gates; FP32 24×24 multiplier = 3200 gates. The QF8 multiply path is 3.5× **to** 48× **cheaper** than alternatives.

QF8-Medium MAC Unit

The recommended design point: 6-bit barrel shifter and 32-bit carry-save Kulisch accumulator for intra-block accumulation, with block-pair results reduced via FP32 arithmetic (matching MX-format accelerator practice).

Table 4: QF8 multiply-path gate count.

Component	Gates	Area (μm^2, 7nm)
Sign XOR	1	0.04
7-bit CLA adder	50	2.0
Overflow/saturation detect	5	0.2
Zero detection	10	0.4
Total: multiply	**66**	**2.6**

Table 5: QF8-Medium MAC unit breakdown (7nm, 1 GHz).

Component	Gates	Area (μm^2)
Multiply path (7-bit log adder)	50	2.0
exp2 LUT (16×12-bit ROM)	100	4.0
6-bit barrel shifter	150	6.0
32-bit carry-save Kulisch accumulator	300	12.0
Control/zero-detect/mux	50	2.0
Total	**650**	**26.0**

Master Comparison

Table 6: Per-MAC unit comparison at 7nm, 1 GHz.

Metric	INT8	FP8 E4M3	FP16	FP32	**QF8-Med**
Gate count	750	1,350	2,400	5,950	**650**
Area (μm^2)	30	54	96	238	**26**
Energy (pJ/MAC)	0.040	0.070	0.130	0.350	**0.033**
Pipeline stages	2	3	3	3–4	**2**
Critical path (gates)	24–30	35–45	38–48	55–70	**20–26**

Systolic Array Scaling (128×128)

The Break-Even Analysis

The log-domain advantage scales differently with bit-width because the multiplier is $O(n^2)$ while the adder is $O(n)$:

The crossover is at $\sim$7–8 bits. Below 7 bits, the LUT and accumulator overhead exceeds multiplier savings. Above 8 bits, log-domain wins decisively and the advantage grows quadratically. This confirms Johnson's 2018 finding that 16-bit ELMA achieves 41% power reduction over FP16.

The Honest Bottom Line

- **QF8 vs FP32/FP16:** Clear, large, robust win (3–10$\times$ savings).
- **QF8 vs FP8:** Clear win ($\sim$2$\times$ area and power savings).

Table 7: QF8-Medium savings ratios vs. each baseline.

Metric	vs FP32	vs FP16	vs FP8	vs INT8
Area savings	**89%**	**73%**	**52%**	**13%**
Power savings	**91%**	**75%**	**53%**	**17%**
Latency improvement	55–65%	35–45%	25–35%	∼5–10%

Table 8: 128×128 systolic array estimates at 7nm, 1 GHz.

Metric	INT8	FP8	FP16	FP32	**QF8-Med**
Compute area (mm^2)	0.49	0.88	1.57	3.90	**0.43**
Total power (W)	0.68	1.19	2.21	5.93	**0.56**
TOPS/W	48.2	27.6	14.8	5.5	**58.5**
TOPS/mm^2	66.9	37.3	20.9	8.4	**76.2**

- **QF8 vs INT8:** Slight win (0.87× area, 0.83× power). The carry-save Kulisch design tips the balance in QF8's favour.
- **The real QF8 value is 2× precision-per-octave at INT8-equivalent hardware cost and half the cost of FP8.**

Training Results

TinyGPT-2 Experiment Configuration

Results

Key Result

Model	Final Train	Final Val	Δ_{val} vs FP32
FP32	2.5388	2.5450	—
FP8 E4M3	2.5401	2.5478	+0.0028 (+0.11%)
QF8	**2.5388**	**2.5445**	**−0.0005** (−0.02%)

QF8 matches FP32 validation loss to within noise (−0.02%), while FP8 E4M3 incurs a small penalty (+0.11%).

Training Loss Trajectory

Both QF8 and FP8 track the FP32 loss curve closely throughout training. Selected validation loss checkpoints:

Benchmark Results

Random matrix multiplication accuracy (measured vs. float64 ground truth):

Table 9: Log-domain break-even across bit widths.

Bit-width	Mult saved (gates)	LUT overhead (gates)	Net	vs INT-N
4	38	120	−82	1.5× worse
6	130	180	−50	1.2× worse
8	**364**	**300**	+64	0.92× **(slight win)**
12	1,145	500	+645	0.63×
16	2,925	800	+2,125	0.40×

Table 10: Training configuration.

Parameter	Value
Model	TinyGPT-2 ($d_{\text{model}} = 128$, heads = 4, layers = 2, $d_{\text{ff}} = 512$)
Vocabulary	256 (byte-level), seq_len = 128
Training	500 steps, batch = 4, lr = 3×10^{-4}, cosine decay
Quantization	Block-scaled round-trip + STE, block_size = 32
Device	CPU

The +6.6 dB advantage is consistent across all tested distributions and matrix sizes, confirming that it is a structural property of the format rather than a distribution-specific artifact.

Literature Positioning

Relationship to Johnson (2018)

The closest prior work is Johnson's "Rethinking Floating Point for Deep Learning" (arXiv:1811.01721), which proposed ELMA: an 8-bit format using log-domain multiply with Kulisch accumulation. QF8's core mechanism — log-domain multiply via integer addition with linear accumulation — *is* Johnson's ELMA.

QF8 should be framed as building on Johnson's architecture with three improvements: (a) fixed-width encoding for hardware simplicity, (b) block scaling for precision/range separation, (c) formal minimax optimality analysis.

Relationship to MX Formats (OCP Microscaling)

QF8 borrows the E8M0 block scaling philosophy directly from the OCP MX specification (Rouhani et al., 2023). The difference is the per-element encoding: MXFP8 uses IEEE-like $4e + 3m$ (8 levels/octave); QF8 uses a log code (16 levels/octave). **QF8 is "MX block scaling + logarithmic per-element encoding."**

Relationship to NF4 (QLoRA)

Both NF4 (Dettmers et al., 2023) and QF8 exploit distribution-aware encoding. NF4 places levels at Gaussian quantiles (optimal for a specific distribution); QF8's log-uniform spacing is minimax-optimal across all distributions. NF4 is a storage format (dequantize before compute); QF8 is a computational format (arithmetic in log domain).

Table 11: Validation loss at selected steps.

Step	FP32	FP8 E4M3	QF8
1	5.3532	5.3568	5.3520
100	2.7647	2.7656	2.7645
250	2.5749	2.5753	2.5753
375	2.4425	2.4443	2.4430
500	2.5450	2.5478	2.5445

Table 12: Matrix multiplication SQNR across sizes.

Size	bfloat16	float32	FP8 E4M3 (block)	**QF8**
$16 \times 32 \times 16$	52.3 dB	139.0 dB	28.6 dB	**35.3 dB**
$64 \times 128 \times 64$	52.5 dB	133.6 dB	28.4 dB	**35.1 dB**
$128 \times 256 \times 128$	52.6 dB	130.8 dB	28.5 dB	**35.1 dB**

Logarithmic Number Systems

QF8 is a descendant of the Logarithmic Number System (LNS), known since the 1970s (Kingsbury & Rayner, 1971; Swartzlander & Alexopoulos, 1975). The fundamental idea of log-domain multiplication is not new. QF8's novelty is the specific combination of format design, block scaling, and formal optimality analysis for ML workloads.

What Is Genuinely Novel

1. **The equalization property (Theorem 3.5, Step 1).** The log-uniform quantizer achieves $\text{MSRE} = \varepsilon^2/12$ for *any* source density f. This density-independence follows from the exact x^2 cancellation in the NMSE ratio — a structural property absent in all prior companding results. Smith [15] observed amplitude-independent SNR for log-PCM, but his result is about signal level, not source distribution, and his distortion formula remains distribution-dependent.

2. **Fixed-rate minimax optimality.** As a consequence of (1), log-uniform uniquely minimizes worst-case NMSE over all source densities. Sun & Goyal [16] prove log companding is optimal for variable-rate MSRE quantization but show it is *not* generally optimal for fixed-rate. Theorem 3.5 provides the complementary fixed-rate minimax result — which holds precisely *because* of (1). No prior 8-bit ML format has any proof of optimality.

3. **The specific format:** `u3.4` log code + E8M0 block scaling + $O(1)$ bit-trick decode.

4. **Quantified advantage:** +6.6 dB / 4.6× NMSE improvement over FP8 E4M3 at identical storage cost (8.25 bits/element). The advantage is mathematical: it follows from 16 vs. 8 levels per octave.

5. **Hardware cost analysis:** 3.5× cheaper multiplier than FP8 (66 vs. 231 gates).

The One-Sentence Pitch

> *QF8 is what you get when you take Johnson's log-domain multiply architecture, replace the posit tapered encoding with a simpler fixed-width log code, add MX-style block scaling,*

Table 13: SQNR across value distributions.

Distribution	FP8 E4M3 (block)	QF8	QF8 advantage
$\mathcal{N}(0, 0.02^2)$ — weights	31.4 dB	38.2 dB	+6.7 dB
$\mathcal{N}(0, 1)$ — post-LN acts	31.6 dB	37.9 dB	+6.3 dB
$\text{LogN}(0, 1)$ — gradients	31.5 dB	38.3 dB	+6.8 dB
$\text{Laplace}(0, 0.02)$ — sparse	31.5 dB	38.0 dB	+6.5 dB
90% sparse + $\mathcal{N}(0, 1)$	31.7 dB	38.3 dB	+6.6 dB

Table 14: QF8 vs. Johnson's ELMA.

Feature	Johnson ELMA (8-bit)	QF8
Log fractional bits	∼6 (posit tapered)	4 (fixed `u3.4`)
Encoding	Posit regime bits (variable)	Fixed-width `u3.4`
Block scaling	None (self-scaled)	E8M0 per 32 elements
LUT size	64 entries (2^6)	16 entries (2^4) — 4× smaller
Optimality proof	None	Minimax NMSE (Theorem 3.5)

and prove that the resulting format is minimax-optimal — yielding 2× *the precision per octave of FP8 at comparable hardware cost.*

Future Work

Scaling Validation

The TinyGPT-2 result (2 layers, 500 steps) demonstrates format viability but says little about behavior at scale. Priority next steps:

1. GPT-2 Small (124M parameters, 300k steps) — does the advantage hold?
2. GPT-2 Medium (355M) — do any instabilities emerge?
3. Comparison against MXFP8 with per-tensor scaling.

Gradient Quantization

QF8's `u3.4` encoding gives 8 octaves per element, which may be insufficient for gradients (which need wider dynamic range than weights/activations). A `u4.3` variant (4 integer + 3 fractional bits = wider range, lower precision) for backward passes is a natural extension but has not been validated. FP8 training uses E5M2 for gradients for the same reason.

BMD Decoder Conjecture

Conjecture 10.1 (BMD Decoder Characterization)**.** *The class of monotone BMD decoders from B-bit integers to positive reals is exactly the class of functions expressible as $\hat{D}(n) = \text{float_reinterpret}(a \cdot n + b)$ for integer constants a, b, or compositions of a bounded number of such reinterpretations with integer arithmetic.*

This remains unproven and should be either proved rigorously or clearly labeled as a conjecture in any paper.

Appendix: Rate-Distortion Context

Gaussian Source Rate-Distortion

For $X \sim \mathcal{N}(0, \sigma^2)$:

$$R(D) = \frac{1}{2}\log_2\left(\frac{\sigma^2}{D}\right), \quad D \leq \sigma^2$$

This yields the **6 dB/bit rule**: each additional bit reduces MSE by 4× (6.02 dB).

Scalar Quantization Gap (Zador)

The maximum improvement from scalar to infinite-dimensional vector quantization:

$$\frac{G_1}{G_\infty} = \frac{\pi e}{6} \approx 1.42 \quad (1.53 \text{ dB})$$

This modest gap means a well-designed scalar quantizer like QF8 comes within 1.53 dB of the VQ optimum.

Bit Allocation

For tensor types $t \in \{w, a, g\}$ with sensitivity weights c_t and distribution variances σ_t^2, the optimal allocation minimizes: $\sum_t c_t \sigma_t^2 \cdot 2^{-2B_t}$ subject to $\sum_t B_t = B_{\text{total}}$.

Solution (reverse water-filling):

$$B_t^* = \frac{B_{\text{total}}}{|\mathcal{T}|} + \frac{1}{2}\log_2\left(\frac{c_t\sigma_t^2}{G}\right), \qquad G = \left(\prod_t c_t \sigma_t^2\right)^{1/|\mathcal{T}|}$$

With corrected parameters ($c_w = 1, c_a = 1, c_g = 10^{-6}$), the near-zero gradient allocation ($B_g = 0.18$) reflects that $c_g = \eta^2 \approx 10^{-6}$ dramatically reduces gradient sensitivity, motivating stochastic rounding or shared-exponent compression for gradients.

References

[1] Johnson, J. (2018). "Rethinking Floating Point for Deep Learning." arXiv:1811.01721.

[2] Micikevicius, P. et al. (2022). "FP8 Formats for Deep Learning." arXiv:2209.05433.

[3] Rouhani, B. et al. (2023). "Microscaling Data Formats for Deep Learning." arXiv:2310.10537.

[4] Dettmers, T. et al. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs." arXiv:2305.14314.

[5] Gustafson, J. & Yonemoto, I. (2017). "Beating Floating Point at its Own Game: Posit Arithmetic."

[6] Horowitz, M. (2014). "Computing's Energy Problem (and what we can do about it)." ISSCC.

[7] Jouppi, N. et al. (2017). "In-Datacenter Performance Analysis of a Tensor Processing Unit." ISCA.

[8] Kingsbury, N. & Rayner, P. (1971). "Digital filtering using logarithmic arithmetic." Electronics Letters.

[9] Swartzlander, E. & Alexopoulos, A. (1975). "The Sign/Logarithm Number System." IEEE Trans. Computers.

[10] Miyashita, D. et al. (2016). "Convolutional Neural Networks using Logarithmic Data Representation." arXiv:1603.01025.

[11] Sze, V. et al. (2017). "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." Proc. IEEE.

[12] Kuzmin, A., Van Baalen, M., Ren, Y., Nagel, M., Peters, J., Blankevoort, T. (2022). "FP8 Quantization: The Power of the Exponent." arXiv:2208.09225.

[13] Bennett, W.R. (1948). "Spectra of Quantized Signals." *Bell System Technical Journal* 27(3):446–472.

[14] Panter, P.F. and Dite, W. (1951). "Quantization Distortion in Pulse-Count Modulation with Nonuniform Spacing of Levels." *Proceedings of the IRE* 39(1):44–48.

[15] Smith, B. (1957). "Instantaneous Companding of Quantized Signals." *Bell System Technical Journal* 36(3):653–709.

[16] Sun, J.Z. and Goyal, V.K. (2011). "Scalar Quantization for Relative Error." *Proc. IEEE Data Compression Conference (DCC)*, pp. 293–302.

[17] Gray, R.M. and Neuhoff, D.L. (1998). "Quantization." *IEEE Trans. Information Theory* 44(6):2325–2383.

[18] Gersho, A. and Gray, R.M. (1992). *Vector Quantization and Signal Compression.* Kluwer Academic.